

Paper 3 Session Handoff v9.0

PAPER 3 — SESSION HANDOFF v9.0

Session date: 7 August 2026 **Read with:** `PAPER3_HANDOFF_v8_0.md` (pipeline state), Papers 1 & 2, Berith’s Frontiers paper, and `OBLIQUE_CONTACT_FINDINGS_v1.md`.

This document covers ONE working session. It does not repeat v8.0’s description of the pipeline — only what changed, what was learned, and what is still open.

0. THE THIRTY-SECOND VERSION

Two separate things were achieved today.

Scientifically: the tactile simulation was characterised across pad roll and rod tilt. It reads 0° and 90° contacts almost perfectly and **fails on diagonals** — not by reporting a wrong angle, but by producing a *rounder blob* than the geometry demands, at which point PCA reports an arbitrary near-axis-aligned direction. This is the sharpest form the tilt finding has taken and it is ready to send to Berith.

Engineering: the motion pipeline was repaired end to end. Pad rolls of 45° and 90°, which previously could not execute at all, now work. The pre-check and the executor agree because they finally plan the same routes. A blocked motion is now detected and refused instead of grinding into the object and reporting success. The gripper is in the collision model.

Nothing here is yet lab-safe. See §7.

1. FILES CHANGED THIS SESSION

Replace these; everything else is untouched.

file	what changed
<code>viz/stitching.py</code>	rolled-pad drawing + shared roll geometry
<code>viz/validation.py</code>	rolled round-trip sampler
<code>viz/blob_axis.py</code>	geometric expected-angle model, band-width fit, no-contact floor
<code>viz/heatmaps.py</code>	third panel: pad pose on the rod + robot visit order
<code>main_gui.py</code>	blob-axis button, band-width box, reachability checkbox, pt00 drawn purple
<code>sim/collect_from_config.py</code>	everything in §4
<code>examples/scenes/ur5e_gripper.yml</code>	NEW — gripper collision spheres

`examples/scenes/ur5e.yml` (Berith’s) is **deliberately untouched**.

2. NEW ENVIRONMENT VARIABLES

All have safe defaults; the GUI’s command works unmodified.

variable	default	effect
<code>GRASP_ENABLE_GRAPH</code>	1	free move searches joint space (falls back to local)
<code>GRASP_APPROACH_ALONG_TOOL</code>	1	descend along the tool axis, not world Z
<code>GRASP_COLLISION_WORLD</code>	1	table slab + object in the collision world
<code>GRASP_OBJECT_COLLISION</code>	1	object is an obstacle during free moves
<code>GRASP_CONTACT_ZONE_M</code>	0.030	object released only this close to the grasp
<code>GRASP_TOOL_COLLISION</code>	0	1 → use <code>ur5e_gripper.yml</code> (gripper checked)
<code>GRASP_STEP_SHORTFALL_MM</code>	8.0	descent aborts if it falls this far behind
<code>GRASP_MAX_RESIDUAL_MM</code>	3.0	fingers refuse to close beyond this error
<code>GRASP_REACH_STRICT</code>	0	1 → Jacobian conditioning is fatal again
<code>GRASP_TOOL_PROBE</code>	0	1 → measure tool geometry, write <code>tool_extents.json</code>
<code>GRASP_IK_PROBE</code>	1	probe failed points with cuRobo’s IK solver
<code>GRASP_ROBOT_YAML</code>	—	explicit robot description override

A warning learned the hard way: the multi-line command with `\` continuations repeatedly broke when pasted, silently dropping the env vars and running with defaults. The tell is `--config: command not found` at the end of the log. **Paste as ONE line**, and always

verify the first console lines match what you intended.

3. THE SCIENCE — OBLIQUE CONTACT

3.1 The expected angle was rebuilt (this matters)

The original `expected = roll - tilt` was wrong twice:

- **Sign.** `blob_axis`'s `x` runs with the COLUMN INDEX, but `_taxel_centers` puts column 0 at the *largest* across-coordinate. So `blob_axis`'s `x` is MINUS the pad's across axis and any hand-derived angle comes out negated.
- **The line is not the blob.** What the taxels see is the contact line CLIPPED by the 22×37 window. Push the pad off-centre and a tilted band runs off the edge, leaving a short stub whose principal axis is NOT the line's angle.

`expected` is now built geometrically: the contact band is rasterised onto the real taxel footprints via `stitching._taxel_centers` and put through the SAME `blob_axis`. No coordinate is transformed by hand, so the sign cannot be wrong, and the 7×4 quantisation is automatically included.

Band width = 15.0 mm, fitted from an upright run's across-pad profile (residual 0.0082, ≈2.4 mm indentation on the Ø26 rod). Use 15.0 for every run so the series stays comparable. Some reports below were computed at the old 8.0 default — the angles barely move but `cover` does.

3.2 The estimator is not the explanation

`blob_axis.metric_selftest()` on a PERFECT straight line — no sensor, no CNN:

true	20°	25°	30°	35°	45°
PCA reads	20.7	24.4	27.9	31.1	38.1

Unbiased below ~25°, ~11% loss at 35°. **A measured angle far below these cannot be blamed on the metric.**

3.3 PAD ROLL series — rod upright (STRONG evidence)

Coverage 57–100%, both sensors agree.

pad roll	expected	s1 measured	s1 error	meas elong	exp elong	cover
0°	0.00	−1.25	−1.25	3.14	3.31	100%
15°	−16.4	−4.2	+12.2	—	—	—
30°	−27.00	−5.82	+21.18	1.88	2.94	86%
45°	−38.38	+3.41	+41.79	1.34	2.08	71%
45° (repeat)	−36.39	−1.11	+35.27	1.41	2.60	79%
60°	−49.99	−2.58	+47.40	1.18	2.08	71%
75°	−67.00	+89.81	−23.19	2.12	1.75	57%
90°	−89.8	+89.7	−0.5	—	—	—

The measured angle stays near 0° from 15° to 60°, then snaps to ~90° between 60° and 75°. It never reports a diagonal. Both endpoints are read correctly. The 45° error is repeatable across two independent runs and both sensors.

3.4 ROD TILT series — pad upright (supporting only)

The offsets had to change per angle — the rod pivots about its centre, so a point 50 mm up its axis swings sideways by 50·sin θ (35 mm at 45°); fixed offsets would leave the pad off the rod entirely. Consequence: coverage 32–50%, so this series is NOT directly comparable to §3.3.

rod tilt	expected	s1 meas	s1 err	s2 meas	s2 err	cover
15°	+5.81	−1.34	−7.15	+7.58	+1.77	39%
30°	+21.59	+22.62	+1.03	+21.96	+0.37	43%
45°	+48.46	−10.09	−58.55	+7.19	−41.27	50%
60°	+70.04	+19.69	−50.35	+30.92	−39.11	43%
75°	+83.64	−0.04	−83.69	−89.20	+7.16	32%

Two honest caveats: **rod 30° reads correctly** (+22.6 vs +21.6) — a genuine ~22° diagonal resolved properly, a real counter-example to “diagonals are never resolved”. And **s1/s2 disagree by up to 66°** here, which does not happen in the pad-roll series and is unexplained.

3.5 The finding that survives BOTH series

Measured blobs are systematically **rounder** than the geometry demands, worst where the contact is most diagonal:

series	angle	meas along	exp along	ratio
pad	0°	3.14	3.31	0.95
pad	30°	1.88	2.94	0.64
pad	45°	1.34	2.08	0.64
pad	60°	1.18	2.08	0.57
rod	15°	3.30	4.06	0.81
rod	45°	1.19	1.83	0.65
rod	75°	1.12	3.58	0.31

The failure is loss of the ridge, not a mis-angled ridge. Where a diagonal band should appear, a rounder blob appears and PCA then reports an arbitrary direction. Consistent with a network trained only on axis-aligned normal indentation — but that is a hypothesis, not a demonstrated cause.

4. THE ENGINEERING — WHAT WAS BROKEN AND HOW

Each of these was found by measurement, not guesswork.

4.1 Rolled pads were painted right but drawn and sampled wrong

The splat already rotated (4 Aug). Four *other* places still assumed an upright 22×37 footprint: column-3 repaint, column-4 dashed frame, `_composite_extended`'s extension numbers, and `validation_sample_canvas`.

Evidence: the old sampler returned **SSIM 0.53** where the truth is 1.00; the old outline contained only **85.5%** of painted cells. Extension numbers were overstated by 6.8 mm in Y and 2.9 mm in Z at −25° roll.

Fix: one shared definition (`pad_roll_deg`, `is_flat`, `pad_half_extents`, `pad_corners`, `rotated_footprint_index`) used by the splat, every drawing, and the inverse sampler. Upright runs verified **pixel-identical**.

4.2 `pad_target_world_mm` was wrong at roll

Computed as `ee_z - TOOL_OFFSET_Z`, valid only upright. Off by 54 mm at 20° and **221 mm at 90°**. Now derived from the rotated offset vector.

4.3 45° and 90° pad rolls could not execute — TWO causes

(a) The planner never searched. `plan_free_move` used `enable_graph=False` — local optimisation only. The IK probe found the 45° pose at **0.001 mm** error but at `shoulder_pan +1.96 rad` while the arm sat at −0.85: reachable, but ~160° away in joint space, which a local optimiser cannot cross. → graph search first, local as fallback.

(b) The descent contradicted itself. `plan_stitched_z` stepped the target along **world Z**, but `CASE13_WEIGHT = [1,1,1,1,1,0]` with `project_pose_to_goal_frame=True` frees the **tool's z**. Upright these coincide; at 90° they are perpendicular, so the planner was asked to travel along an axis it was not allowed to move along. Measured gap: **141 mm**. → descend along the tool axis.

With both fixed, 0/15/30/45/60/75/90° all execute.

4.4 The pre-check proved a path the executor never drives

`_ik_at` tried a **direct** plan first; the executor always goes up-then-down (first point) or pad-to-pad (later points). Every report carried certified via the DIRECT plan only. → `_ik_at` now plans the executor's actual route. `precheck_path` reads `up_then_down` / `pad_to_pad`.

4.5 The Jacobian gate was flapping and silently dropping points

`evaluate_reachability` walks a straight line in joint space — a motion the executor never performs. The same 90° pose reported `min_sigma` 0.0060 → 0.0040 → 0.0012 → 0.0026 across identical runs and flipped to `cond(J)=1749 > 1000` on one of them, while cuRobo planned to it and IK hit it to 0.002 mm.

→ conditioning and dry-run non-convergence are now **advisory** (`reason_code: ok_advisory`, listed in `advisory[]`). Hard checks (joint limits, frozen joints, delta bounds) remain fatal. `GRASP_REACH_STRICT=1` restores the old veto.

4.6 A blocked descent closed the fingers anyway

At 90° roll with `pad_offset_y = +80` the gripper had to pass through the rod. Every stitched step landed short, **47 mm** accumulated, the trim failed, and the fingers **closed anyway** — while the ledger recorded `exec_stage: complete`, `1/1 grasps OK`. The gripper shoved the cylinder across the table.

Root cause: `CASE13_WEIGHT` frees the travel axis, so a step can report success while barely moving.

→ Two guards. Each step now measures how far it actually travelled along the axis; cumulative shortfall past 8 mm aborts with `the path is BLOCKED`. And if the residual still exceeds 3 mm after the trim, the fingers **do not close** — `exec_stage: descent_residual`, recorded as a FALSE POSITIVE.

Verified: on the real 47.3 mm case this trips at **step 2**.

4.7 Collision was blind past the flange

ur5e.yml’s tool0 had ONE sphere of radius **−0.01** (negative = disabled). cuRobo’s model stopped at the flange: gripper, fingers and pads did not exist for collision. At 90° roll the flange passes 157 mm clear of the rod, so sweeping the *gripper* through it looked perfectly safe.

→ ur5e_gripper.yml : 18 spheres for coupler + gripper body + knuckles, plus forearm_link ignoring tool0 and self_collision_buffer: tool0 = 0 (the stock 0.025 inflated the spheres into constant self-collision).

Fingers and pads are deliberately NOT modelled. To grasp a 26 mm rod the pads must wrap around it, so any honest model of them overlaps the object at contact — they can never be collision-checked against the thing they exist to touch. Attempts to include them made every grasp unreachable. For the same reason no sphere may enclose the tool axis forward of ~110 mm.

4.8 Phase-dependent object collision

The rod cannot be a plain obstacle (it is the grasp target) but must be one during transit — at 90° roll the “descent” is a 100 mm **horizontal** sweep that went straight through it.

→ object enabled during free moves, released only inside CONTACT_ZONE_M (30 mm) of the goal.

4.9 Diagnostics added

- execution_ledger.json — predicted vs actual for every point, with agreement / FALSE POSITIVE / FALSE NEGATIVE / skipped counts.
- Reason codes: ok, ok_advisory, ik_no_solution, path_infeasible, singularity, joint_limit, manual_limit, collision_static.
- IK probe on failed points: four queries (rolled / upright / unrolled flange / no object) with a plain-language verdict.
- tool_extents.json (GRASP_TOOL_PROBE=1) — measured tool geometry.
- No-contact floor in blob_axis — a noise-only map used to yield a confident angle; on the 35-point grid it invented −8 to −12° for thirteen grasps that never touched the rod.
- heatmaps.py third panel — pad pose on the rod, robot visit order, skipped points.

5. MEASURED TOOL GEOMETRY (keep — hard to re-derive)

From probe_tool_extents, in the tool0 frame, mm past the flange:

part	z	x	y
coupler	−3 ... 14	±37.9	±37.9
gripper base	7 ... 101	±37.6	±42.5
knuckles	66 ... 121	±17.6	±6.7...56.9
fingers (OPEN)	102 ... 130	±13.5	±44.9...74.8
sensor pads	121 ... 165	±14.2	±41.4...62.8

Cross-check: pad centre at **156.6 mm** (pad_truth_probe.json) sits inside the measured 121-165 mm pad box. Stock ur5e.yml covers **51 mm**; ur5e_gripper.yml covers **133 mm**.

6. VERIFICATION RUNS (all 7 Aug, all one-point)

config	flags	result
upright, y=0 z=40	stock	complete, blob −1.8° / −0.3°
upright, y=0 z=40	TOOL_COLLISION=1	complete — gripper model does not break it
90°, y=0 z=40	stock	complete, blob +89.2° / −89.8°
90°, y=0 z=40	TOOL_COLLISION=1	complete , blob +89.5° / −89.9°
90°, y=+80	stock	descent blocked at 47.4 mm → abort, fingers stayed open , 1 FALSE POSITIVE
90°, y=+80	TOOL_COLLISION=1	rejected in the PRE-CHECK, no motion at all

The last two rows are the point of the whole exercise: the same pose that drove into the cylinder is now refused before the arm moves.

7. STILL OPEN

Not lab-safe yet. The collision world contains only a table slab and the rod — no fixture, pedestal or cables. Fingers and pads are not modelled, so a finger *tip* alone catching something is not seen.

The IK probe’s verdict misattributes when TOOL_COLLISION=1: it said “the flange swing from the roll” where the truth was the gripper hitting the rod, because rolled_no_object also fails (self-collision limit) and cannot isolate the object. Cosmetic; the rejection itself is correct.

`min_sigma` **drifts across identical runs** (0.0060 → 0.0012 → 0.0026 on the same pose), implying the arm's start state is not identical between runs. No longer blocking (advisory), but it undermines run-to-run repeatability claims.

Not yet done: 65°/70° to locate the 0→90 snap precisely; a 2D grid where the stitched-map blob axis becomes meaningful; the s1/s2 disagreement in the rod-tilt series; and the frame check inside `probe_tool_extents`, which mixes base-frame spheres with a world-frame flange transform and prints nonsense.

8. NEXT STEPS, IN ORDER

8.1 Blocking — the science

1. **Message Berith** (draft written 7 Aug, see §11). Two asks: object meshes for new shapes, and the diagonal-contact question.
2. Run **65° and 70°** pad roll to pin down where the 0→90 snap happens.
3. Explain the **s1/s2 disagreement** in the rod-tilt series (up to 66°).

8.2 Unblocks the dataset

4. **Swap the cylinder for another object** — prove the process before committing to a big run. Start with a **cuboid**: its contact is a flat patch, so the expected map is trivial and it cleanly tests “can I change the object at all”. Sphere next (small round patch, no axis). *Mostly free*: the scene already reads `shape/diameter_mm/length_mm` from the config and the collision obstacle is built from the same numbers. *Not free*: `blob_axis.expected_patch_map` models a **cylinder's** contact generatrix; for other shapes it must be rewritten, and for non-cylinders the metric should become patch overlap (IoU / SSIM / centroid), not angle.
5. **Collection speed**, before scaling up — but **profile first**. At 35 points the blob FIGURE was 6.6 s of ~8 s total, so the bottleneck may not be where expected. Then consider batching, headless, Calcul Québec.
6. **Review how single points and the stitched map are prepared** while the dataset is still small enough to change cheaply.

8.3 GUI — small, quick

7. Colour pt00 differently in the grid preview **DONE 7 Aug** — purple `#7b2f8e`, 2.8 px, own legend entry; reachability preserved via linestyle.
8. **Fix the palm indicator**, which draws in the wrong place when the pad is rolled.
9. **Show designed grid vs actual executed pad poses** (from `pose_history.json`) so placement error is visible per point.

8.4 Grid frame for rolled pads

10. Add the option to step along the **pad's own axes** instead of world X/Y when the pad is rolled. Scoped earlier as one edit to the pivot-correction block in `collect_from_config.py` plus a checkbox — the stitcher needs no change, because it paints wherever the pad actually was. **Why it matters**: with the pad rolled, stepping in world Y/Z tiles a square in the WORLD but a sheared parallelogram in the PAD's frame, so the swept region is not a clean rectangle of pad-relative coverage. A rolled grid collected without this is quietly wrong and would have to be thrown away rather than re-plotted. **Priority**: after the object swap. It only matters once rolled data is actually being collected.

8.5 Block 3

11. **Resume collection and training** for the **axis-aligned regime** — upright rod, upright pad, which is the Papers 1–2 setting and the bulk of the intended dataset. **Do NOT put rolled-pad or tilted-rod grasps into a training set** until §8.1 is answered: a completion model trained on maps where diagonal ridges render as round blobs would learn the renderer's failure, and no downstream metric would reveal it.
12. Run a **2D grid**, where the stitched-map blob axis finally becomes meaningful (on a 1×N sweep the stitched blob's shape is set by the stepping direction, not the contact).

8.6 Lab safety — later

13. Add the **fixture and pedestal** to the collision world.
14. Decide how to handle **finger tips**, still unmodelled.

8.7 A design question worth settling early

For the intended pairing — *initial pad pose + its tactile map* as input, *stitched map* as output — pad rotation is probably **not needed**. The model learns “given this imprint, complete the surrounding surface”; rolling the pad mainly adds contact orientations, which is exactly what the CNN cannot render faithfully. Rotating the **rod** instead gives orientation variety with the pad upright, so the grid stays clean. If rolled data is collected anyway, item 10 becomes mandatory.

9. THINGS THAT WILL SAVE THE NEXT SESSION TIME

- **Verify the console before every run.** `robot description: , GRASP_APPROACH_ALONG_TOOL = , in-plane spin , USE_COLLISION_WORLD = ,` Several hours were lost to commands that silently ran with defaults.
- **Paste commands as one line.** `--config: command not found` at the end of a log means the env vars were dropped.
- **Use band width 15.0** in the blob-axis box, always.
- **Read `elong` before `error`.** Below ~1.5 the blob is round and its angle means nothing — several dramatic-looking errors are

just that.

- `gripper_modelled` and `tool_collision` are recorded in every report, so any dataset states what was actually collision-checked.
- Isaac Sim swallows `print()` in script mode for some paths — diagnostics that must survive go to files.

10. OPEN DESIGN QUESTIONS (no code needed, decide before collecting)

Does the dataset need rolled-pad grasps at all? See §8.7 — probably not for the intended input→output pairing, and rotating the rod is the cleaner way to get contact-orientation variety.

“Centred grid, n steps per side” means $(2n+1)^2$ points, not n^2 — $n=1$ gives 9, $n=2$ gives 25, $n=3$ gives 49. Unchecked, pt00 is a corner and the grid grows into one quadrant only; checked, pt00 is the centre with equal coverage in all four directions, which is what extrapolation wants.

Tilting the ROD is fine for grid geometry; rolling the PAD is not. With an upright pad the world X/Y steps stay square in the pad frame regardless of rod tilt (only the offsets need adjusting so the pad stays on the object). Rolling the pad shears the swept region — hence item 10 in §8.4.

11. THE MESSAGE TO BERITH (drafted 7 Aug, not yet sent)

Two asks, in this order — the mesh request is easy to say yes to, and the finding lands better as a puzzle than a verdict.

- 1. Object meshes.** Does he have other object meshes for the scene, and what does a new object need in order to be wired up?
- 2. The diagonal-contact question.** Framed as: with the pad rolled against an upright rod, 0° reads within 1° , 90° within 0.5° , but 45° should read -38° and reads $+3^\circ$. The blob is the wrong *shape* rather than the wrong angle — round instead of an elongated ridge, ~35% less elongated than geometry implies. Then the question: could this be a training-data effect (indenters pressed straight down, nothing oblique in the set), or is there another explanation?

Include the self-test line. “If I feed my PCA a perfect synthetic 45° line on a 7×4 grid it returns 38° , so the method itself only loses $\sim 7^\circ$.” This closes off the obvious first response — *are you sure your measurement is right?* — before it is asked.

Attach the 45° and 90° blob figures side by side. Same rod, same pad, same code; one works and one does not. More persuasive than any table. Do NOT send only 90° figures — those show the pipeline working, not the problem.

11.1 A distinction that caused confusion — keep it straight

Three different numbers, easily conflated:

number	what it is
45°	what the geometry says the contact line is
38°	what YOUR PCA returns on a <i>perfect synthetic</i> image at 7×4 — no sensor, no CNN, no Isaac. The resolution limit of 28 taxels.
$+3^\circ$	what Berith’s CNN in Isaac actually reported

`expected` in the blob report is built the same way as the 38° figure, so it already carries that resolution loss — which is precisely why comparing `measured` against `expected` is a fair comparison. The self-test (`blob_axis.metric_selftest()`, or `python3 blob_axis.py x --selftest`) exists only to calibrate the ruler; it is **text output, not a plot**, and it is never data.

11.2 How the “perfect” map is built

Compute where the rod’s contact line falls on the pad from pad pose + rod pose, give it a **15 mm width** with a Hertzian profile, clip it to the 22×37 window, and integrate over each of the 28 taxel footprints. That is `expected_patch_map()`. The 15 mm came from fitting the band width against a real upright run’s across-pad profile (residual $0.0082 \approx 2.4$ mm indentation on the $\varnothing 26$ rod).

12. WHAT TO UPLOAD TO THE NEXT CHAT

This document, `PAPER3_HANDOFF_v8_0.md`, `OBLIQUE_CONTACT_FINDINGS_v1.md`, Papers 1 & 2, Berith’s paper, and the current `collect_from_config.py`, `main_gui.py`, `stitching.py`, `validation.py`, `blob_axis.py`, `heatmaps.py`, `ur5e_gripper.yml`.

Ask the new session to verify file freshness by grepping for `GRASP_TOOL_COLLISION` in `collect_from_config.py` and `TOOL COLLISION MODEL v4` in `ur5e_gripper.yml` before trusting anything above.